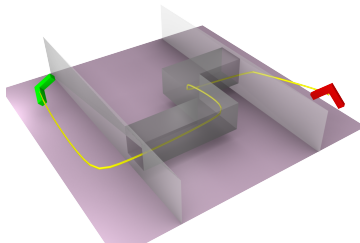
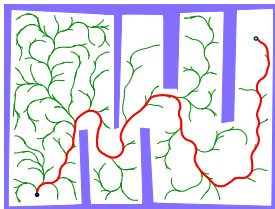
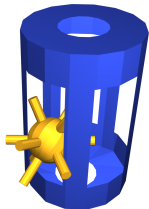
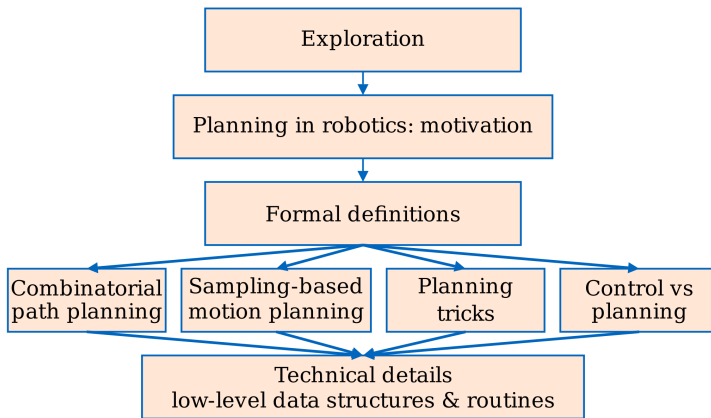


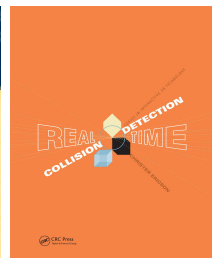
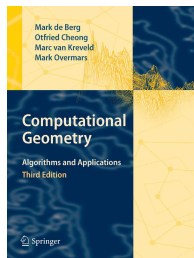
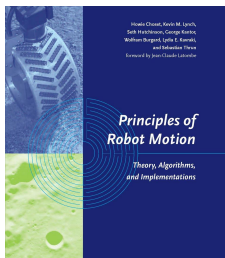
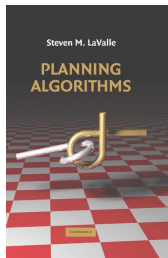
Motion planning: basic concepts

Vojtěch Vonásek

Department of Cybernetics
Faculty of Electrical Engineering
Czech Technical University in Prague







- S. M. LaValle, Planning algorithms, Cambridge, 2006, [online: planning.cs.uiuc.edu](http://online.planning.cs.uiuc.edu)
- H. Choset, K. M. Lynch et al., Principles of Robot Motion: Theory, Algorithms, and Implementations (Intelligent Robotics and Autonomous Agents series), Bradford Book, 2005
- M. de Berg, Computational Geometry: Algorithms and Applications, 1997
- C. Ericson. Real-time collision detection. CRC Press, 2004.

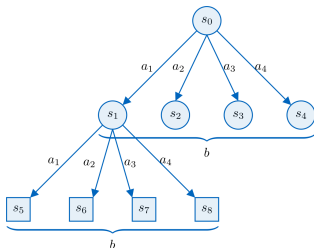
Informal definition: Motion planning is about automatic finding of ways how to move an object (robot) while avoiding obstacles (and considering other constraints)

- “Piano mover’s problem”
- Mathematics: graph theory & topology
- Computational geometry: collision detection
- Computer graphics: visualizations
- Many applications in robotics and even beyond



Terminology

- State: a set of important variables of the system
- Action: how to change the state

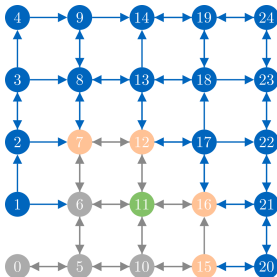


Classic state-space search

- Iteratively expand known states using actions
- Avoid already visited ones
- Terminate if goal state is found
- BFS, DFS, Dijkstra, ...
- Suitable only for discrete spaces/graphs

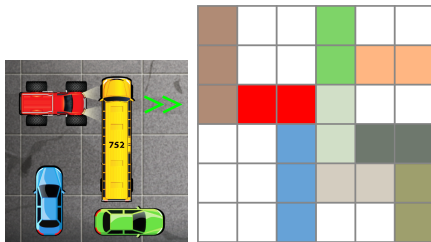
Breadth-first search (BFS)

- Insert initial state s_0 into the queue and mark it as visited
- While the queue is not empty:
 - $s_n = \text{pop from the queue}$
 - If s_n is goal state, return path from s_n to s_0
 - For each action a applicable at s_n :
 - $s = f(s_n, a)$ — apply the action
 - If s is not visited, insert it into the queue and mark as visited
- If the queue is empty, return no solution



Breadth-first search (BFS)

- Insert initial state s_0 into the queue and mark it as visited
- While the queue is not empty:
 - $s_n = \text{pop from the queue}$
 - If s_n is goal state, return path from s_n to s_0
 - For each action a applicable at s_n :
 - $s = f(s_n, a)$ — apply the action
 - If s is not visited, insert it into the queue and mark as visited
- If the queue is empty, return no solution



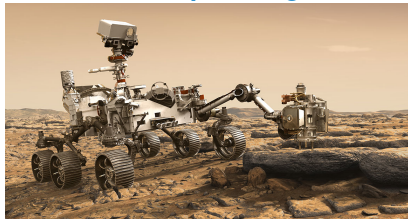
Rush hour puzzle

Breadth-first search (BFS)

- Insert initial state s_0 into the queue and mark it as visited
- While the queue is not empty:
 - $s_n = \text{pop from the queue}$
 - If s_n is goal state, return path from s_n to s_0
 - For each action a applicable at s_n :
 - $s = f(s_n, a)$ — apply the action
 - If s is not visited, insert it into the queue and mark as visited
- If the queue is empty, return no solution

Classic state-space search cannot be used for motion planning

- Robot positions are continuous
- Motion actions are continuous
- Discretization is not practically suitable



World $\mathcal{W}$

- is space where the robot operates
- $\mathcal{W}$ is usually $\mathcal{W} \subseteq \mathbf{R}^2$ or $\mathcal{W} \subseteq \mathbf{R}^3$
- $\mathcal{O} \subseteq \mathcal{W}$ are obstacles

Robot $\mathcal{A}$

- $\mathcal{A}$ is the geometry of the robot
- $\mathcal{A} \subseteq \mathbf{R}^2$ (or $\mathcal{A} \subseteq \mathbf{R}^3$)
- or set of links $\mathcal{A}_1, \dots, \mathcal{A}_n$ for n -body robot

Configuration q

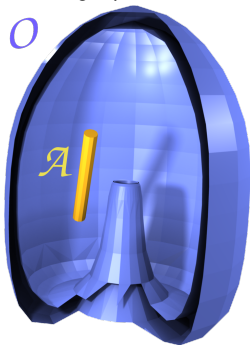
- Specifies position of **every** point of $\mathcal{A}$ in $\mathcal{W}$
- Usually a vector of **Degrees of freedom (DOF)**

$$q = (q_1, q_2, \dots, q_n)$$

Configuration space $\mathcal{C}$ (aka C-Space or $\mathcal{C}$ -space)

- $\mathcal{C}$ is a set of **all** possible configurations

3D Bugtrap benchmark



$$\mathcal{W} \subseteq \mathbf{R}^3, \mathcal{A} \subseteq \mathbf{R}^3$$
$$\mathcal{O} \subseteq \mathbf{R}^3$$

(x, y, z) is 3D position

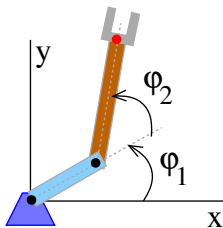
(r_x, r_y, r_z) is 3D rotation

$$q = (x, y, z, r_x, r_y, r_z)$$

$\mathcal{C}$ -space is 6D

- A configuration is a **point** in $\mathcal{C}$
- $\mathcal{A}(q)$ is set of **all points** of the robot determined by configuration $q \in \mathcal{C}$
- Therefore, point $q \in \mathcal{C}$ **fully** describes how the robot looks in $\mathcal{W}$
- $\mathcal{C}$ has as many dimensions as robot's DOFs
- $\mathcal{C}$ is considered “high-dimensional” if number of DOFS > 4

Example: a robotic arm with two revolute joints; $q = (\varphi_1, \varphi_1) \rightarrow 2D$ $\mathcal{C}$ -space
Robot geometry has two rigid shapes: $\mathcal{A}_1$ and $\mathcal{A}_2$



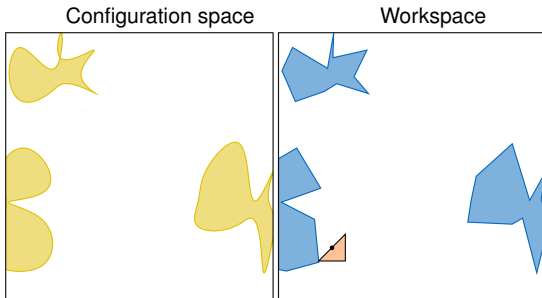
Obstacles in the configuration space

$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}, \quad \mathcal{C}_{\text{obs}} \subseteq \mathcal{C}$$

- $\mathcal{C}_{\text{obs}}$ contains robot-obstacle collisions and self-collisions
- Self-collisions: robotic arms, tethered robots
- Collision-free configurations

$$\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$$

- $q \in \mathcal{C}_{\text{free}} \rightarrow q$ is feasible

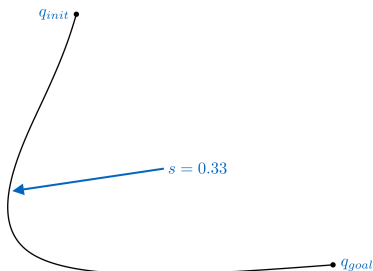


- A **path** in $\mathcal{C}$ is a continuous curve connecting $q_{\text{init}} \in \mathcal{C}$ and $q_{\text{goal}} \in \mathcal{C}$:

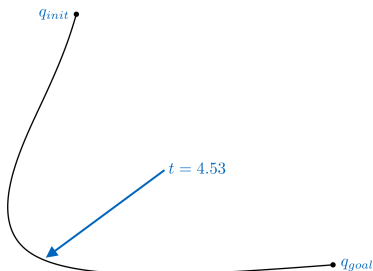
$$\tau : s \in [0, 1] \rightarrow \tau(s) \in \mathcal{C}; \quad \tau(0) = q_{\text{init}} \text{ and } \tau(1) = q_{\text{goal}}$$

- A **trajectory** is a path parameterized by time

$$\tau : t \in [0, T] \rightarrow \tau(t) \in \mathcal{C}$$



Path



Trajectory

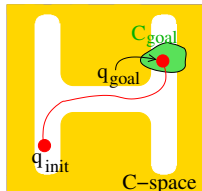
Given: model of the world $\mathcal{W}$ and robot $\mathcal{A}$, start $q_{\text{init}} \in \mathcal{C}_{\text{free}}$, and goal region $\mathcal{C}_{\text{goal}} \subseteq \mathcal{C}_{\text{free}}$

Path planning

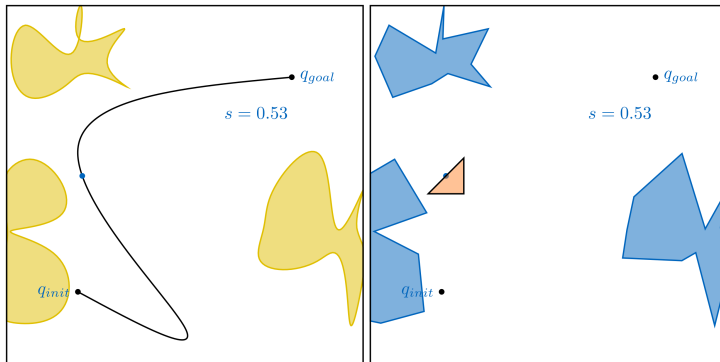
- To find a collision-free path $\tau(s)$ from q_{init} to $\mathcal{C}_{\text{goal}}$ such that $q(s) \in \mathcal{C}_{\text{free}}$ for all $s \in [0, 1]$, $s(0) = q_{\text{init}}$, $s(1) \in \mathcal{C}_{\text{goal}}$

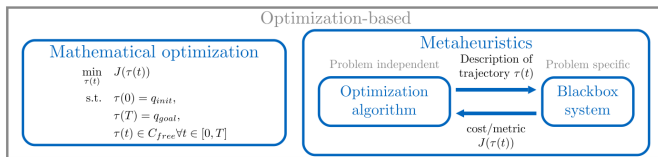
Motion planning

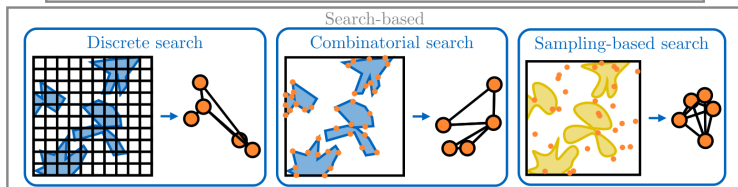
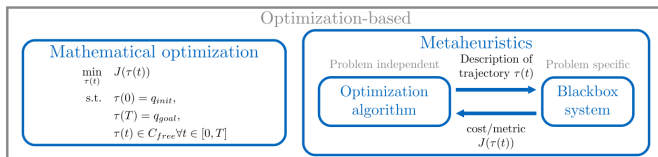
- To find a collision-free trajectory $\tau(t)$ from q_{init} to $\mathcal{C}_{\text{goal}}$ such that $q(t) \in \mathcal{C}_{\text{free}}$ for all $t \in [0, T]$, $s(0) = q_{\text{init}}$, $s(T) \in \mathcal{C}_{\text{goal}}$
- The above definition is considered as **feasible path/motion planning**
- Using $\mathcal{C}_{\text{goal}}$ instead of single $q_{\text{goal}} \in \mathcal{C}_{\text{free}}$ is more practical
- No optimality criteria is considered



A trajectory/path in $\mathcal{C}$ defines motion in workspace







Optimization-based

Mathematical optimization

$$\begin{aligned} \min_{\tau(t)} \quad & J(\tau(t)) \\ \text{s.t.} \quad & \tau(0) = q_{init}, \\ & \tau(T) = q_{goal}, \\ & \tau(t) \in C_{free} \forall t \in [0, T] \end{aligned}$$

Metaheuristics

Problem independent

Description of trajectory $\tau(t)$

Problem specific

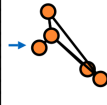
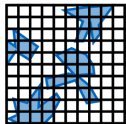
Optimization algorithm

Blackbox system

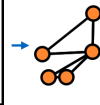
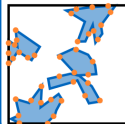
cost/metric $J(\tau(t))$

Search-based

Discrete search



Combinatorial search

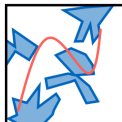
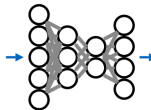


Sampling-based search



Learning-based motion planning

Neural networks for planning



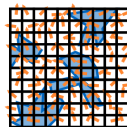
Policy search

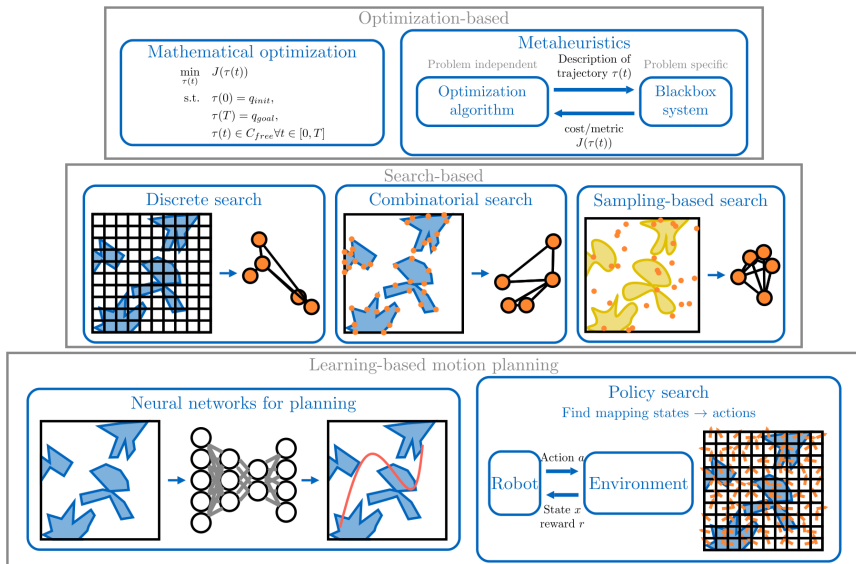
Find mapping states $\rightarrow$ actions

Robot

Action a
State x
reward r

Environment





- ARO course focuses on search-based methods

Main idea

- Motion planning is formulated as an optimization problem

$$\begin{aligned} \min_{\tau(t)} \quad & J(\tau) \\ \text{subject to} \quad & \tau(0) = \mathbf{q}_{\text{init}}, \\ & \tau(T) = \mathbf{q}_{\text{goal}}, \\ & \tau(t) \in \mathcal{C}_{\text{free}}, \quad \forall t \in [0, T] \end{aligned}$$

- J is a weighted sum of desired properties, e.g.

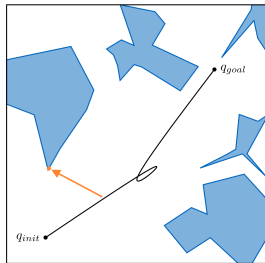
$$J(\tau) = \underbrace{w_l R_l(\tau)}_{\text{length}} + \underbrace{w_s R_s(\tau)}_{\text{smoothness}} + \underbrace{w_o R_o(\tau)}_{\text{obstacle avoidance}} + \dots$$

- The crucial part is the cost of avoiding obstacles $R_o(\tau)$
- $J(\tau)$ must be smooth (for some optimization frameworks)

Naïve obstacle cost

$$R_o(\tau) = \min_{q \in \tau} d_o(q)$$

- $d_o(q)$: distance from q to the nearest obstacle
- Not smooth where the closest obstacle features changes



distance d_o

Smooth obstacle cost

$$R_o(\tau) = \int_0^T \text{dist}_o(\tau(t)) dt$$

$$\text{dist}_o(q) = \begin{cases} (d_{\text{safe}} - d_o(q))^2 & \text{if } d_o(q) < d_{\text{safe}} \\ 0 & \text{otherwise} \end{cases}$$

- $\text{dist}_o(q)$ is zero if q is farther than d_{safe} from obstacles

Discrete optimal planning

- Previous formulation assumed continuous trajectory $\tau(t)$
- Often, trajectory is represented as a sequence of states

Robot motion model

- Robot motion is described by $q(k+1) = f(q, u)$
 - where u is a control input
 - $u \in \mathcal{U}$ or $u_{\min} \leq u \leq u_{\max}$

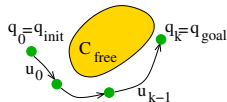
Discrete path/trajectory

- Path/trajectory is defined by the sequence of inputs

$$\pi_k = (u_0, \dots, u_{k-1})$$

- By applying $q_1 = f(q_0, u_0), q_2 = f(q_1, u_1), \dots$ we get the state sequence

$$(q_0, q_1, \dots, q_{k-1}, q_k)$$



Discrete optimal planning

- The problem of finding path/trajectory is solved by finding the control sequence π_k

$$\min_{\pi_k=(u_0,\dots,u_{k-1})} L(\pi_k)$$

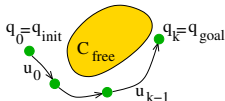
$$\text{subject to } q_{i+1} = f(q_i, u_i), i = 0, \dots, k-1$$

$$q_0 = q_{\text{init}}$$

$$q_k = q_{\text{goal}}$$

$$q_k \in \mathcal{C}_{\text{free}}$$

$$u_{\min} \leq u_i \leq u_{\max}, \quad i = 0, \dots, k-1$$



- where $L(\pi_k)$ is the cost

$$L(\pi_k) = l_f(q_k) + \sum_{i=0}^{k-1} l(q_i, u_i)$$

- the final term $l_f(q_k) = 0$ if $q_k = q_{\text{goal}}$; it is ∞ otherwise
- $L(\pi_k) = \infty \rightarrow$ infeasible solution
- $L(\pi_k) < \infty \rightarrow$ a feasible solution

How is optimal control defined?

- Optimal control for a discrete-time (and finite horizon)
- initial state is x_i , goal state x_n may be given (or not)

$$\min_{u_i, \dots, u_{N-1}, (x_i), \dots, x_n} \left(\phi(x_n, N) + \sum_{k=i}^{N-1} L_k(x_k, u_k) \right)$$

subject to

$$x_{k+1} = f_k(x_k, u_k)$$

$$u_{lb} \leq u_k \leq u_{ub}$$

$$x_{lb} \leq x_k \leq x_{ub}$$

equations by Z. Hurak: Discrete-time optimal control — direct approach (lectures notes of ORR)

Optimization-based planning vs control

- Optimal motion planning and optimal control are the same problems
- Both can provide path/trajectory between two states

Optimization-based planning vs control

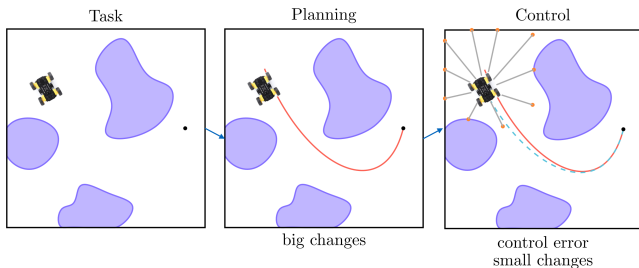
- Optimal motion planning and optimal control are the same problems
- Both can provide path/trajectory between two states

Motion planning

- Explicitly account for obstacles
- Many methods to find feasible (yet non-optimal) solutions
- Suitable for large-scale environments
- Slow for control

Control

- Trajectory is achieved via mathematical optimization
 - we need the gradient ∇J and analytic distance to obstacles
- In control theory, obstacles are (typically) not considered
- Suitable for following reference trajectory
- Not suitable for large-scale environments with obstacles



- Global plan delivered by motion planning
- Sensing (actual position, speed, etc.) controlled along planned path
- i.e., errors in actuation are handled by control
- Replanning when global change occurs (e.g. new obstacle that cannot be handled by control)

Does not make sense to solve motion plan by control-theory methods

Does not make sense to control via planning!

How to formulate and solve optimal planning

- Linear programming, mixed integer linear programming¹²
- Nonlinear programming, quadratic/sequential quadratic programming³
 - ∇J is required
 - can be trapped in local minima (the path cannot "skip" obstacles)
 - computing d_O is slow for multiple obstacles non-convex obstacles (no analytical solution)
 - a good initialization is required

How to find the “good initialization” ?

- Using other planning paradigms e.g., search-based, machine learning-based or metaheuristics

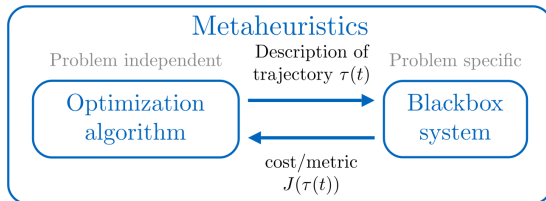
¹D. Ioan et al. “Mixed-Integer Programming in Motion Planning”. In: *Annual Reviews in Control* 51 (2021), pp. 65–87. DOI: [10.1016/j.arcontrol.2020.10.008](https://doi.org/10.1016/j.arcontrol.2020.10.008).

²M. Srinivasan et al. “Fast Multi-Robot Motion Planning via Imitation Learning of Mixed-Integer Programs”. In: *IFAC-PapersOnLine* 54.20 (2021), pp. 598–604. DOI: [10.1016/j.ifacol.2021.11.237](https://doi.org/10.1016/j.ifacol.2021.11.237).

³Z. Zhao et al. “A survey of optimization-based task and motion planning: From classical to learning approaches”. In: *IEEE/ASME Transactions On Mechatronics* 30.4 (2024), pp. 2799–2825.

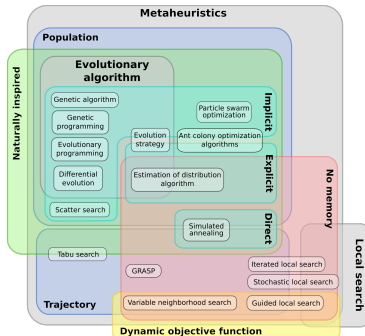
Metaheuristics

- General methods for searching a solution
- They are not problem specific, therefore
 - Gradient ∇J is not required, J may be a blackbox
- But using domain knowledge is welcome
- Usually stochastic and approximate
- Genetic algorithms/Evolutionary methods, simulated annealing, ...



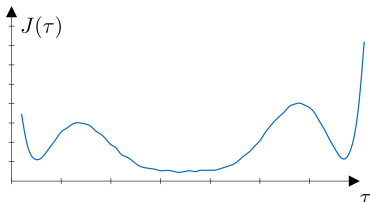
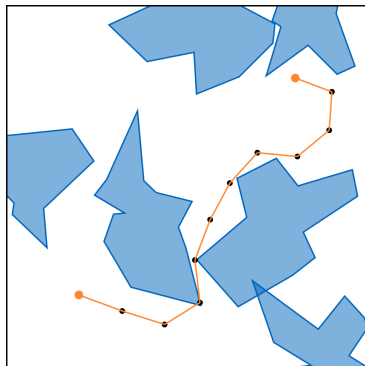
Metaheuristics

- General methods for searching a solution
- They are not problem specific, therefore
 - Gradient ∇J is not required, J may be a blackbox
- But using domain knowledge is welcome
- Usually stochastic and approximate
- Genetic algorithms/Evolutionary methods, simulated annealing, . . .



Metaheuristics

- General methods for searching a solution
- They are not problem specific, therefore
 - Gradient ∇J is not required, J may be a blackbox
- But using domain knowledge is welcome
- Usually stochastic and approximate
- Genetic algorithms/Evolutionary methods, simulated annealing, . . .



Metaheuristics

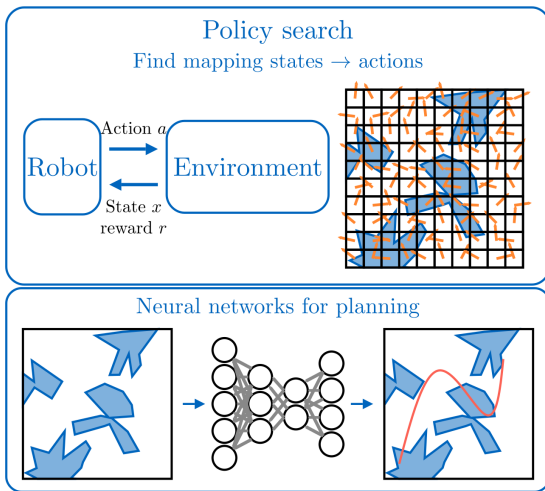
- General methods for searching a solution
- They are not problem specific, therefore
 - Gradient ∇J is not required, J may be a blackbox
- But using domain knowledge is welcome
- Usually stochastic and approximate
- Genetic algorithms/Evolutionary methods, simulated annealing, ...

More details

- A0M33EOA - Evoluční optimalizační algoritmy

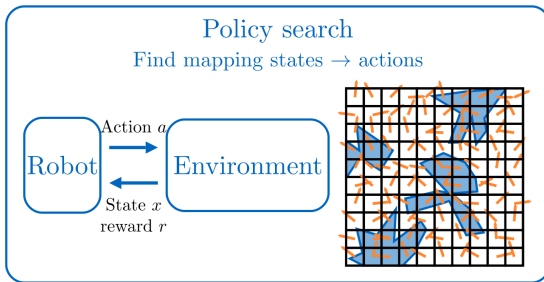
Main idea

- Automatically discover paths/motion from (many) training examples



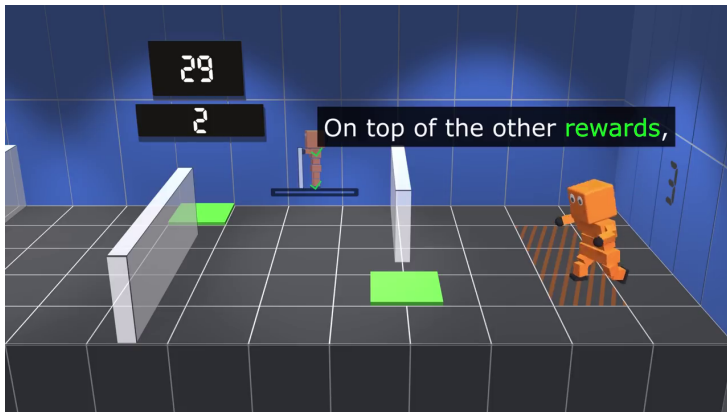
Reinforcement learning

- The goal is to learn policy, i.e. mapping states $\rightarrow$ actions
- Robot learns actions/decisions by interacting with (dynamic) environment that provides rewards
- The policy should maximize total cumulative reward



Reinforcement learning

- The goal is to learn policy, i.e. mapping states $\rightarrow$ actions
- Robot learns actions/decisions by interacting with (dynamic) environment that provides rewards
- The policy should maximize total cumulative reward



Reinforcement learning

- The goal is to learn policy, i.e. mapping states $\rightarrow$ actions
- Robot learns actions/decisions by interacting with (dynamic) environment that provides rewards
- The policy should maximize total cumulative reward

Pros/cons

- Suitable for low-dimensional states
- Requires many training episodes, simulations are necessary
- Proper shaping of reward functions is not trivial⁴⁵

More details

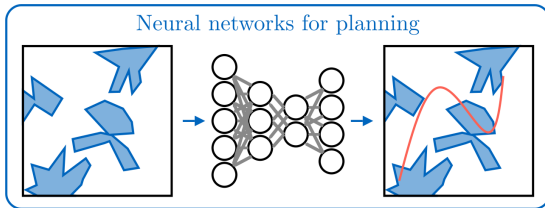
- B3B33UROB — Učni robotu — organized by Karel

⁴S. Ibrahim et al. “Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications”. In: *IEEE Access* 12 (2024), pp. 175473–175500. DOI: [10.1109/ACCESS.2024.3504735](https://doi.org/10.1109/ACCESS.2024.3504735).

⁵B. Huang and Y. Jin. “Reward Shaping in Multiagent Reinforcement Learning for Self-Organizing Systems in Assembly Tasks”. In: *Advanced Engineering Informatics* 54 (Oct. 2022), p. 101800. DOI: [10.1016/j.aei.2022.101800](https://doi.org/10.1016/j.aei.2022.101800).

Planning via neural networks (NN)

- Ideally, NN should provide path/trajectory, but that is difficult
- Often, NN serves as supporting tool to other methods⁶⁷
 - to bias/guide search
 - or to approximate collision detection or learn metric in $\mathcal{C}$
- Many NN-based tools require 2D image of $\mathcal{W}$ $\rightarrow$ only for simple planning scenarios



⁶G. P. De Carvalho et al. “Data-driven motion planning: A survey on deep neural networks, reinforcement learning, and large language model approaches”. In: *IEEE Access* (2025).

⁷T. McMahon et al. “A survey on the integration of machine learning with sampling-based motion planning”. In: *Foundations and Trends® in Robotics* 9.4 (2022), pp. 266–327.

Optimization-based

Mathematical optimization

$$\begin{aligned} \min_{\tau(t)} \quad & J(\tau(t)) \\ \text{s.t.} \quad & \tau(0) = q_{init}, \\ & \tau(T) = q_{goal}, \\ & \tau(t) \in C_{free} \forall t \in [0, T] \end{aligned}$$

Metaheuristics

Problem independent

Description of
trajectory $\tau(t)$

Problem specific

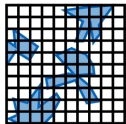
Optimization
algorithm

Blackbox
system

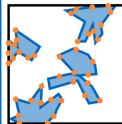
cost/metric
 $J(\tau(t))$

Search-based

Discrete search



Combinatorial search

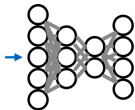


Sampling-based search



Learning-based motion planning

Neural networks for planning



Policy search

Find mapping states $\rightarrow$ actions

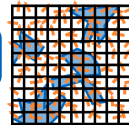
Robot

Action a

Environment

State x

reward r

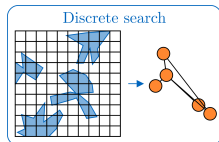


Main idea

- We “convert” continuous space $\mathcal{C}$ into a graph; a path in the graph defines path/motion in $\mathcal{W}$

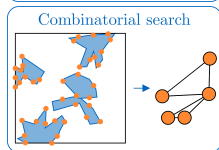
Discrete search

- Use discrete grid to make the graph
- Suitable for low-DOF robots (mobile, Roomba, etc.)



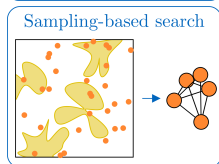
Combinatorial search

- The graph is defined by boundary of obstacles $\mathcal{O}$
- Suitable for low-DOF point/disc robots



Sampling-based search

- The graph is made from random samples in $\mathcal{C}$
- Support many-DOF robots, any-shaped robot, robots moving under constraints
- Most popular in robotics

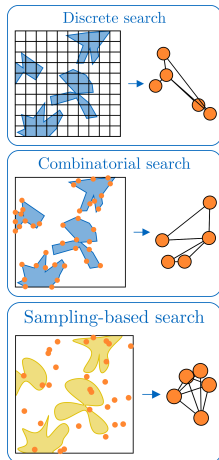


Main idea

- We “convert” continuous space $\mathcal{C}$ into a graph; a path in the graph defines path/motion in $\mathcal{W}$

Advantages of search-based methods

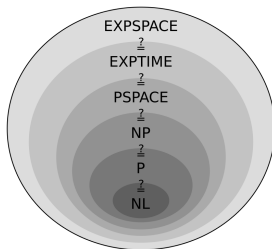
- Easy to understand (very important!)
 - easy for modifications and tuning
- Easier to implement than optimization or ML-based methods
- Some are even optimal and complete
- Suitable for initialization of optimization-based planners
- Do not require tremendous and slow training



- Motion planning is P-Space complete⁸
- To solve motion planning, we have to "know" $\mathcal{C}$, i.e., $\mathcal{C}_{\text{obs}}$ and $\mathcal{C}_{\text{free}}$
- Obstacles $\mathcal{C}_{\text{obs}}$ are given implicitly:

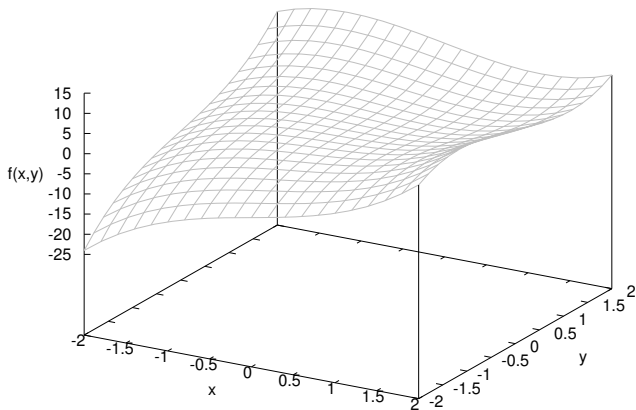
$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}$$

- Implicit definition does not allow to enumerate points/boundary of $\mathcal{C}_{\text{obs}}$
 - it is difficult to obtain distance to obstacles $d_{\mathcal{O}}$



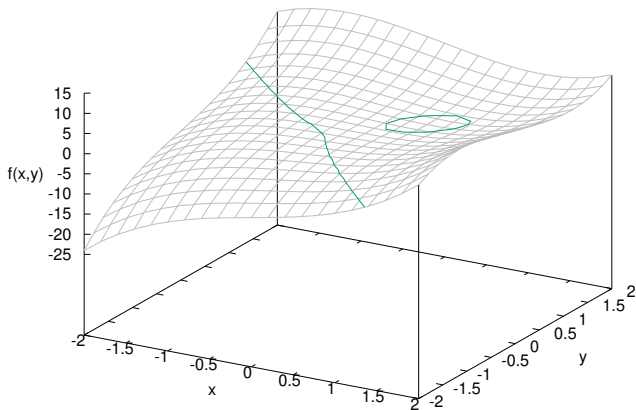
⁸J. F. Canny. *The Complexity of Robot Motion Planning*. Cambridge, MA: MIT Press, 1988.
ISBN: 9780262031363.

$$f(x, y) = x^3 - 2xy + y^3$$



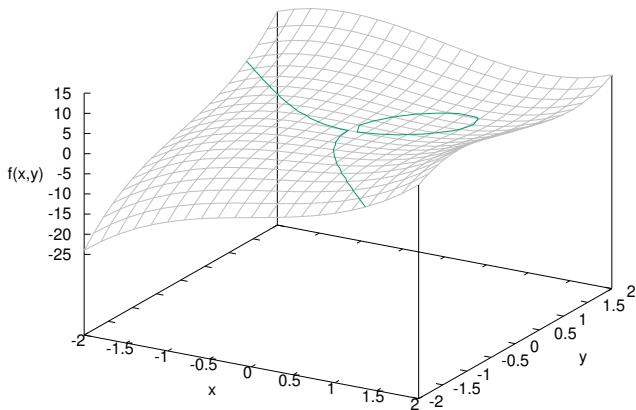
$f(x, y)$

$$f(x, y) = x^3 - 2xy + y^3$$



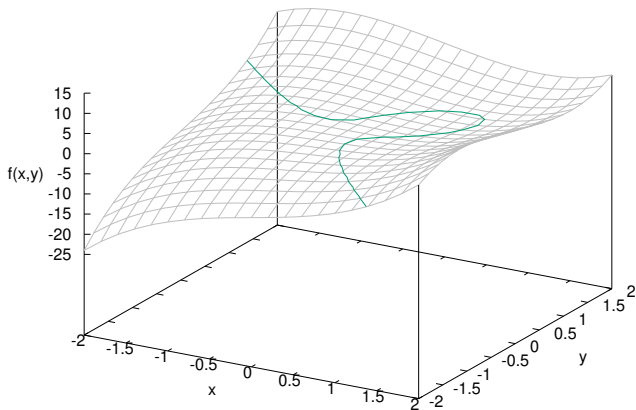
$$f(x, y) = -0.1$$

$$f(x, y) = x^3 - 2xy + y^3$$



$$f(x, y) = 0$$

$$f(x, y) = x^3 - 2xy + y^3$$



$$f(x, y) = 0.1$$

- How to get explicit list of obstacles from the implicit obstacles

$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} \mid \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}, \quad \mathcal{C}_{\text{obs}} \subseteq \mathcal{C}$$

- i.e., how to enumerate points on the border of the obstacles?

Explicit construction of $\mathcal{C}_{\text{obs}}$

- $\mathcal{A}(0)$ is the robot at origin
- $-\mathcal{A}(0)$ is achieved by replacing all $x \in \mathcal{A}(0)$ by $-x$
- Obstacles in $\mathcal{C}$ are determined by the Minkowski sum

$$\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus -\mathcal{A}(0)$$

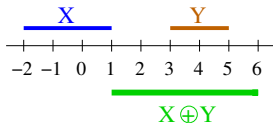
- Theoretical principle, not used in practice (you will see why)

Minkowski sum $\oplus$ of two sets $X, Y \subset \mathbb{R}^n$ is

$$X \oplus Y = \{x + y \in \mathbb{R}^n \mid x \in X \text{ and } y \in Y\}$$

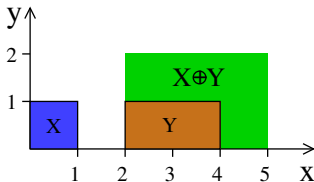
1D example: $X = [-2, 1]$, $Y = [3, 5]$

$$X \oplus Y = [1, 6]$$



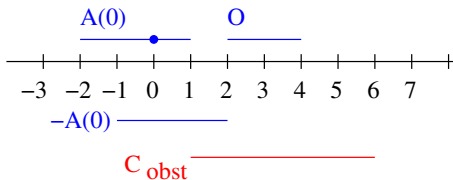
2D example: $X = [0, 1] \times [0, 1]$, $Y = [2, 4] \times [0, 1]$

$$X \oplus Y = [2, 5] \times [0, 2]$$



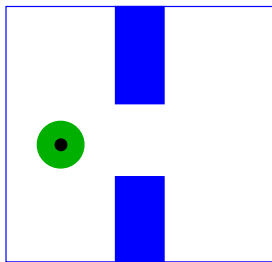
Example: 1D robot $\mathcal{A} = [-2, 1]$ and obstacle $\mathcal{O} = [2, 4]$:

$$\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus -\mathcal{A}(0)$$

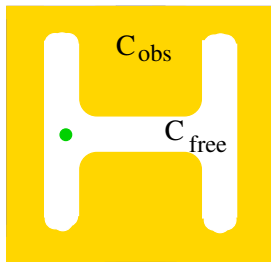


$$\mathcal{C}_{\text{obs}} = [1, 6]$$

- 2D workspace $\mathcal{W} \subseteq \mathbf{R}^2$
- 2D disc robot $\mathcal{A} \subseteq \mathbf{R}^2$, reference point in the disc center
- We assume **only translation** $\rightarrow$ configuration $q = (x, y)$, $\mathcal{C}$ is 2D



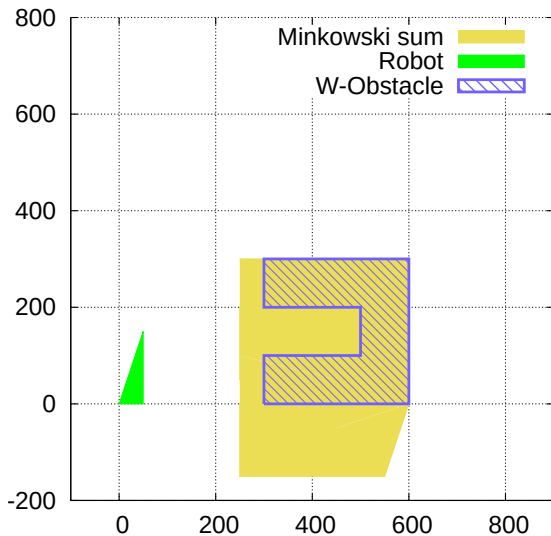
Workspace



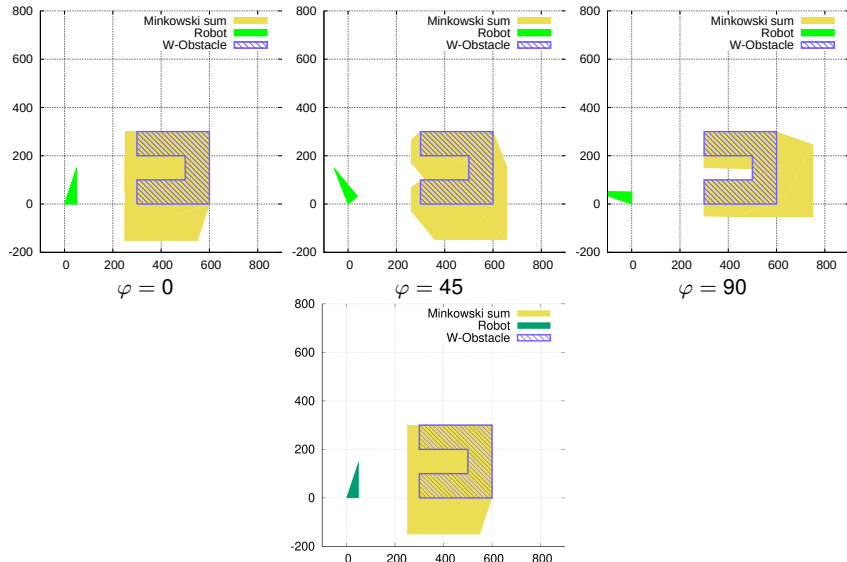
Configuration space

- Volume of $\mathcal{C}_{\text{free}}$ depends both on the robot and obstacles
- What happens if the robot is a point?

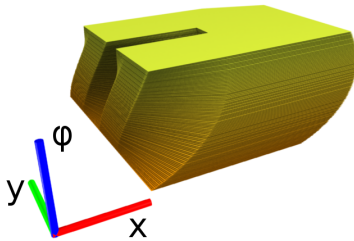
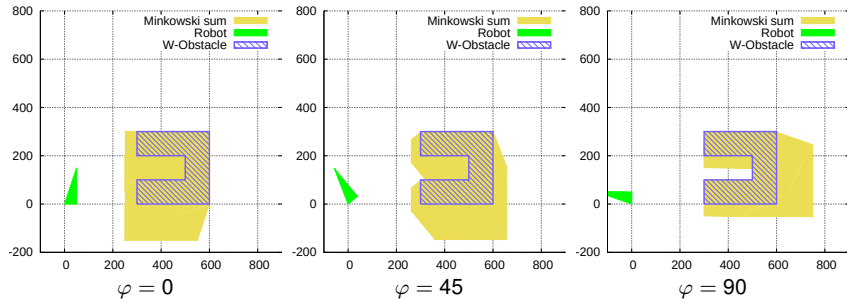
- 2D robot, only translation, $q = (x, y) \rightarrow 2D \mathcal{C}$



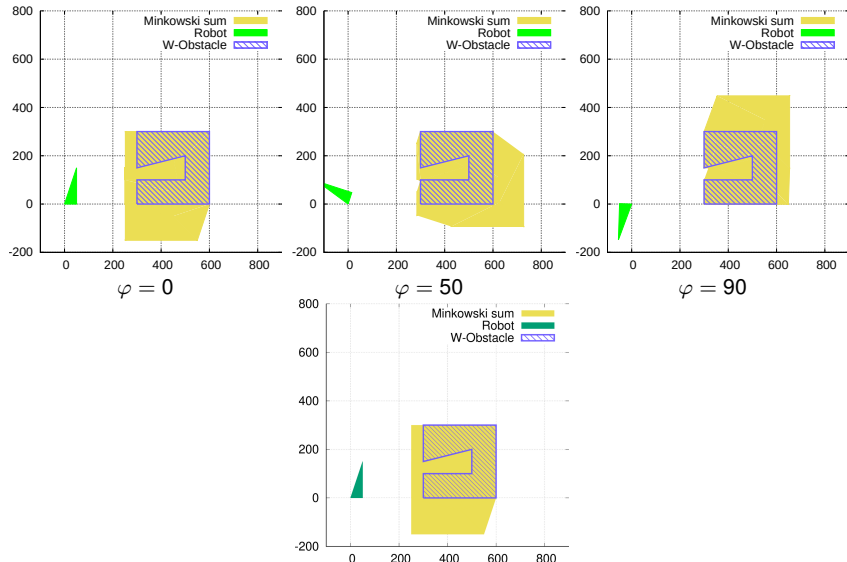
- 2D robot, translation + rotation, $q = (x, y, \varphi) \rightarrow 3D \mathcal{C}$
- Requires to compute Minkowski sum for each rotation



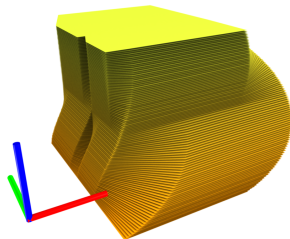
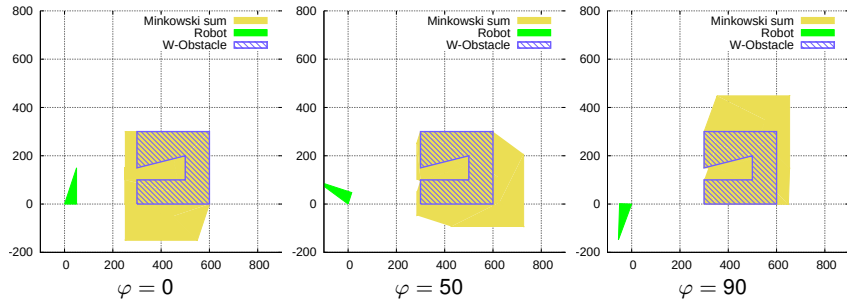
- 2D robot, translation + rotation, $q = (x, y, \varphi) \rightarrow 3D \mathcal{C}$
- Requires to compute Minkowski sum for each rotation



- 2D robot, translation + rotation, $q = (x, y, \varphi) \rightarrow 3D \mathcal{C}$
- Requires to compute Minkowski sum for each rotation



- 2D robot, translation + rotation, $q = (x, y, \varphi) \rightarrow 3D \mathcal{C}$
- Requires to compute Minkowski sum for each rotation



- We have only 2D/3D models of robots and obstacles
 - Making $\mathcal{C}_{\text{obs}}$ from $\mathcal{O}$ using Minkowski is valid only for translation-only systems
 - Other DOFS need to be discretized and Minkowski sum computed for each combination
- Explicit construction of $\mathcal{C}_{\text{obs}}$ from $\mathcal{O}$ and $\mathcal{A}$ using Minkowski sum is generally too difficult, and it is not practically used.

Complexity of Minkowski sum of two objects with n and m elements

2D polygons

- convex $\oplus$ convex, $O(m + n)$
- convex $\oplus$ arbitrary, (mn)
- arbitrary $\oplus$ arbitrary, (m^2n^2)

3D polyhedrons

- convex $\oplus$ convex, $O(mn)$
- arbitrary $\oplus$ arbitrary, (m^3n^3)

Optimization-based

Mathematical optimization

$$\begin{aligned} \min_{\tau(t)} \quad & J(\tau(t)) \\ \text{s.t.} \quad & \tau(0) = q_{init}, \\ & \tau(T) = q_{goal}, \\ & \tau(t) \in C_{free} \forall t \in [0, T] \end{aligned}$$

Metaheuristics

Problem independent

Description of
trajectory $\tau(t)$

Problem specific

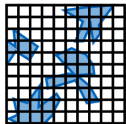
Optimization
algorithm

Blackbox
system

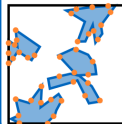
cost/metric
 $J(\tau(t))$

Search-based

Discrete search



Combinatorial search

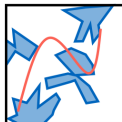
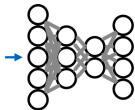


Sampling-based search



Learning-based motion planning

Neural networks for planning



Policy search

Find mapping states $\rightarrow$ actions

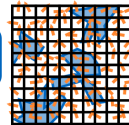
Robot

Action a

State x

reward r

Environment



Completeness

- An algorithm is complete, if for any input it correctly reports in **finite time if there is a solution or not**
- If a solution exists, it **must** return one **in a finite time**
- Computationally very hard (P-Space complete)
- Complete methods exist only for low-dimensional problems

Probabilistic completeness

- Algorithm is prob. complete if for scenarios with an existing solution the probability of finding that solution converges to one
- If solution does not exist, the method can run forever

Optimal vs. non-optimal

- Optimal planning: algorithm ensures finding of the optimal solution (according to a criterion)
- Non-optimal: any feasible solution is returned

Asymptotically optimal

- With increasing runtime, a solution provided by the algorithm converges to the optimal solution